

CAHIER DES CHARGES

Projet : Planification d'Examens par Coloration de Graphes

Niveau L2 Informatique — Théorie des Graphes | Durée : 2 semaines

I. Présentation du projet

Contexte : Le service de suivi des programmes d'un établissement a besoin d'un outil pour organiser les examens de fin de période. La période peut être organisée en trimestre ou en semestre. Une période est constituée d'un ensemble de jours dans lesquels se déroule les examens. Chaque jour est constitué de quarts créneaux horaires dans lesquelles peuvent être affectés les examens. Plusieurs cours (appelés Unités d'Enseignement, UE) doivent être planifiés sur des créneaux horaires et dans des salles, sans qu'il y ait de conflits entre eux.

Votre mission : Utiliser la théorie des graphes plus précisément l'algorithme de coloration de graphe pour résoudre automatiquement ce problème de planification avec le langage Python ou le langage C.

Principe clé : Deux UE qui partagent des étudiants en commun ne peuvent pas avoir lieu au même moment. En graphe, cela correspond à un sommet par UE, et une arête entre deux UE qui ne peuvent pas être au même créneau.

II. Contraintes à respecter

Les contraintes suivantes définissent ce qu'un planning valide doit obligatoirement satisfaire :

2.1 Contraintes obligatoires

- **Un étudiant ne compose qu'une seule UE par créneau :** si deux UE ont des étudiants en commun, elles doivent être placées dans des créneaux différents.
- **Une salle ne peut accueillir qu'un seul examen par créneau :** deux UE simultanées ne peuvent pas utiliser la même salle.
- **La capacité de la salle doit être respectée :** le nombre d'étudiants inscrits à un examen ne doit pas dépasser la capacité de la salle affectée.
- **Un surveillant ne peut pas être dans deux salles en même temps :** les UE partageant un même enseignant-surveillant ne peuvent pas être simultanées.
- **Certaines UE nécessitent un laboratoire informatique :** elles ne peuvent pas être affectées à une salle standard.

2.2 Contraintes souhaitées (amélioration de la qualité)

- **Espacement entre examens d'une même filière :** deux examens appartenant à la même filière ne doivent pas être programmés dans des créneaux consécutifs (au moins un créneau libre entre eux).

- **Équilibre de la charge par créneau** : le nombre d'examens par créneau doit être aussi uniformément réparti que possible.
- **Priorité aux UE à grand effectif** : les UE avec le plus d'étudiants inscrits sont programmées en priorité dans les premiers créneaux.
- **Interdictions explicites entre certaines UE** : certaines paires d'UE ne doivent pas avoir lieu le même jour, indépendamment des inscriptions étudiantes.

III. Tâches à réaliser

Le projet est découpé en trois parties progressives, plus une partie bonus.

Partie 1 — Construction du graphe de conflits

- Représenter chaque UE par un sommet et ajouter une arête entre deux UE si elles partagent au moins un étudiant.
- Implémenter le graphe avec une matrice d'adjacence et une liste d'adjacence.
- Calculer et afficher le nombre de sommets, d'arêtes et le degré de chaque sommet.
- Visualiser le graphe avec les bibliothèques `networkx` et `matplotlib`.

Partie 2 — Algorithmes de coloration

- **Algorithme Welsh-Powell** : trier les sommets par degré décroissant, puis attribuer le plus petit créneau disponible à chaque UE.
- **Algorithme DSATUR** : attribuer les créneaux en choisissant à chaque étape l'UE dont le voisinage est le plus saturé en couleurs distinctes.
- Comparer les deux algorithmes sur plusieurs jeux de données : nombre de créneaux utilisés, temps d'exécution.

Partie 3 — Affectation des salles et planning final

- Après la coloration, affecter chaque UE à une salle en vérifiant les contraintes de capacité et de type de salle (standard ou labo).
- Vérifier automatiquement le respect de toutes les contraintes et afficher un rapport d'audit.
- Générer le planning final sous forme de tableau (créneau $\times$ salle) et l'exporter en fichier CSV.

Partie Bonus — Optimisation. [Optionnel]

- Planter une méthode d'optimisation (recuit simulé ou algorithme génétique) pour réduire encore le nombre de créneaux en tenant compte des contraintes souhaitées.

IV. Livrables attendus

- **Code source Python** organisé en fichiers séparés (graphe, coloration, affectation, planning), commenté et compressé en un fichier ZIP.

- **Rapport PDF (5 à 9 pages)** décrivant la modélisation, les algorithmes utilisés, les résultats obtenus et une analyse critique.
- **Image du graphe coloré** générée par matplotlib (format PNG).
- **Fichier CSV du planning** au format créneau $\times$ salle, avec le code UE et l'effectif dans chaque cellule.

V. Critères d'évaluation

1. **Modélisation et formalisation du graphe** — 6 points
2. **Implémentation et comparaison des algorithmes** — 6 points
3. **Qualité du rapport et de l'analyse** — 4 points
4. **Présentation orale et clarté des explications** — 4 points